Automated E-Commerce Deployment Platform

Graduation Project Documentation

Track: DevOps Engineering

Team size: 5 members

Status: Draft for team review and alignment with official documentation guidelines

Team Members

Team member
Ahmed Elsayed Abd Elfatah
Mostafa Biomy Abd Elsatar Elshplangy
Elsayed Mosaad Elsayed Kamal
Eslam Hosny Elsayed
Kerolos wise Abd ultawab

SECTION 1.0: PROJECT PLANNING & MANAGEMENT

Project Idea & Scope: This project aims to design and build an automated, fully-integrated deployment platform to host e-commerce applications based on a microservices architecture (Web UI, Search Engine, Payment Gateway, and Database). The solution delivers end-to-end operational cycle automation starting from Infrastructure as Code (IaC), configuration management, automated orchestration on Kubernetes with Horizontal Pod Autoscaling (HPA), and real-time observability to ensure Zero-Downtime Deployment.

RESPONSIBILITY ASSIGNMENT MATRIX (RACI)

1. Infrastructure & Config (IaC)
Authoring Terraform scripts for AWS resources (EC2, RDS, S3, Secrets Manager) and Ansible playbooks for one-click environment configuration.

2. Containerization & Orchestration
Building lightweight Docker images for microservices, managing Kubernetes clusters, and configuring HPA and Health Checks.

3. CI/CD Pipeline & Integration
Developing Jenkins pipelines integrated with GitHub, configuring Nginx Reverse Proxy, and enforcing automated rollback strategies.

4. Observability & Documentation
Setting up Prometheus & Grafana dashboards, tracking performance metrics/alerts, and maintaining technical repository docs.

RISK MANAGEMENT & MITIGATION REGISTER

Potential Technical Risk	Impact	Mitigation Strategy
Cloud Overhead Costs	VERY HIGH	Restrict resources to AWS Free Tier limits and utilize local simulated development environments prior to cloud deployment.
Pipeline Build Failure	HIGH	Execute staged automated tests and trigger an instant rollback to the last stable release upon build failure.
Service Outage	MEDIUM	Configure Kubernetes Liveness/Readiness probes to automatically detect, terminate, and recreate unhealthy containers.
Database Data Loss	CRITICAL	Adopt AWS RDS with Multi-AZ deployment enabled and schedule automated periodic backups directly to S3 storage.

SECTION 2.0: LITERATURE REVIEW & PROPOSED SOLUTION

Technical Evaluation Metric	Legacy Monolithic Server	Semi-Automated (EC2 + Docker Compose)	Proposed Solution (Jenkins & GitOps)
Deployment & Updates	Slow manual execution; high human error	Semi-automated; requires admin intervention	Fully automated one-click workflow
Service Continuity	Inevitable downtime during updates	Potential partial downtime during builds	Zero-Downtime Deployment
Scalability Capacity	Expensive vertical scaling; strict limits	Requires manual re-configuration	Horizontal Pod Autoscaling (HPA)
Recovery Resilience	Complex manual recovery taking hours	Manual container restart from backup	Automated Rollback in < 30 seconds

SECTION 3.0: REQUIREMENTS GATHERING & ANALYSIS

Stakeholder Analysis: System requirements were derived from Developers (rapid and smooth deployment), Operations Team (end-to-end visibility and monitoring), Store Management (business continuity and financial loss prevention), and End Consumers (high performance and seamless purchasing).

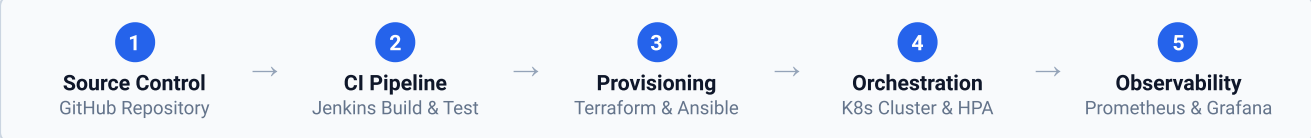
Functional Requirements

- Automated creation/destruction of AWS resources via Terraform.
- Containerization and isolation of Web, Search, and Payment services.
- Traffic routing and load balancing via Nginx and AWS ELB.
- Automated code fetch and build upon GitHub Push events.
- Automated rollback engine execution on test failure.

Non-Functional Requirements

- **Performance:** Response latency < 200ms under heavy load.
- **Availability:** 99.9% uptime with resource redundancy.
- **Security:** Databases isolated inside Private Subnets.
- **Observability:** Real-time dashboards tracking CPU & Memory.
- **Maintainability:** Complete platform redeploy in < 10 mins.

SYSTEM ARCHITECTURE FLOW



SECTION 4.0: PROJECT DELIVERABLES & ARTIFACTS MATRIX

#	Technical Deliverable	Toolstack	Repository Path (GitHub)
1	**Dockerized Microservices**	Docker / Docker Compose	/docker/services/
2	**CI/CD Automated Pipelines**	Jenkins / Declarative Pipeline	/jenkins/Jenkinsfile
3	**Kubernetes Orchestration & HPA**	Kubernetes Manifests	/k8s/manifests/hpa.yaml
4	**Server Configuration Playbooks**	Ansible Automation	/ansible/playbooks/site.yml
5	**Monitoring & Alert Dashboards**	Prometheus & Grafana	/monitoring/dashboards/
6	**Nginx Routing & Reverse Proxy**	Nginx Proxy Engine	/nginx/conf.d/default.conf
7	**IaC Infrastructure Provisioning**	Terraform (EC2, RDS, S3, ELB)	/terraform/main.tf
8	**Automated Rollback Engine**	Shell Scripting & K8s Rollout	/jenkins/scripts/rollback.sh
9	**Central Source Code Repository**	Git & GitHub Platform	github.com/Mostafa Elshplangy/Automated-E-Commerce...

KEY PERFORMANCE INDICATORS (KPIS)

0%	< 30s	99.9%	100%
SERVICE DOWNTIME	AUTO RECOVERY (MTTR)	TARGET AVAILABILITY	IAC & PIPELINE AUTOMATION

SECTION 5.0: FORMAL APPROVAL & SIGN-OFF

This document and all associated technical deliverables uploaded to the official project repository have been thoroughly reviewed and validated for final graduation presentation.

Academic Supervisor Signature: _____
Prof. Dr. [Supervisor Name]

Evaluation Committee Signature: _____
Official Date: October 17, 2026